# Лабораторна робота №3

REST API з Express.js та інтеграційне тестування

Метою лабораторної роботи є набуття практичних навичок розробки RESTful API на Express.js з TypeScript та написання інтеграційних тестів з використанням Supertest

## Завдання 1: Ініціалізація проєкту

Створено структуру проєкту з усіма необхідними залежностями. Сутність — Recipe (рецепт).

package.json:

```
{
  "name": "lab3-recipes-api",
  "scripts": {
    "dev": "ts-node src/server.ts",
    "build": "tsc",
    "test": "jest",
    "test:coverage": "jest --coverage"
  },
  "dependencies": { "express", "cors", "zod" },
  "devDependencies": { "typescript", "ts-node", "jest", "ts-jest",
"supertest", ... }
}
```

Структура проєкту:

```
project/
├── src/
│   ├── schemas/recipe.schema.ts
│   ├── storage/recipe.ts
│   ├── routes/recipe.ts
│   ├── middleware/validate.ts
│   ├── middleware/errorHandler.ts
│   ├── app.ts
│   └── server.ts
├── tests/recipe.test.ts
├── tsconfig.json
├── jest.config.js
└── package.json
```

## Завдання 2: Схеми та типи сутності

Файл: src/schemas/recipe.schema.ts

Сутність Recipe містить: title (1-100 символів), description (до 500), cookingTimeMinutes (1-1440), difficulty (easy/medium/hard), cuisine (ukrainian/italian/...), servings (1-100).

```
import { z } from 'zod';

export const DIFFICULTY_LEVELS = ['easy', 'medium', 'hard'] as const;
export const CUISINE_TYPES = ['ukrainian', 'italian', 'asian', 'american', 'french',
'other'] as const;

export const createRecipeSchema = z.object({
title: z.string().min(1, "Назва обов'язкова").max(100),
description: z.string().max(500).optional(),
cookingTimeMinutes: z.number().int().min(1).max(1440),
difficulty: z.enum(DIFFICULTY_LEVELS),
cuisine: z.enum(CUISINE_TYPES).optional(),
servings: z.number().int().min(1).max(100).optional(),
});

export const updateRecipeSchema = createRecipeSchema.partial();

export type CreateRecipeInput = z.infer<typeof createRecipeSchema>;
export type UpdateRecipeInput = z.infer<typeof updateRecipeSchema>;

export type Recipe = CreateRecipeInput & {
id: string;
createdAt: Date;
updatedAt: Date;
};
```

## Завдання 3: Модуль зберігання (in-memory storage)

Файл: src/storage/recipe.ts

Реалізовано CRUD-модуль на основі Map<string, Recipe> з підтримкою фільтрації та скидання стану.

```
import { randomUUID } from 'crypto';
import { Recipe, CreateRecipeInput, UpdateRecipeInput } from
'../schemas/recipe.schema';

const store = new Map<string, Recipe>();
```

```
export function getAllRecipes(filters?: RecipeFilters): Recipe[] {
  let recipes = Array.from(store.values());
  if (filters?.difficulty) recipes = recipes.filter(r => r.difficulty === fil-
ters.difficulty);
  if (filters?.cuisine) recipes = recipes.filter(r => r.cuisine === filters.cuisine);
  if (filters?.maxCookingTime) recipes = recipes.filter(r => r.cookingTimeMinutes <=
filters.maxCookingTime!);
  if (filters?.search) {
    const term = filters.search.toLowerCase();
    recipes = recipes.filter(r => r.title.toLowerCase().includes(term));
  }
  return recipes;
}

export function createRecipe(data: CreateRecipeInput): Recipe {
  const id = randomUUID();
  const now = new Date();
  const recipe: Recipe = { ...data, id, createdAt: now, updatedAt: now };
  store.set(id, recipe);
  return recipe;
}

export function updateRecipe(id: string, data: UpdateRecipeInput): Recipe | undefined {
  const existing = store.get(id);
  if (!existing) return undefined;
  const updated = { ...existing, ...data, updatedAt: new Date() };
  store.set(id, updated);
  return updated;
}

export function deleteRecipe(id: string): boolean { return store.delete(id); }
export function clearStorage(): void { store.clear(); }
export function getQuickRecipes(maxMinutes = 30) { return getAllRecipes({ maxCooking-
Time: maxMinutes }); }
```

## Завдання 4: CRUD-маршрути та middleware

Файли: src/routes/recipe.ts, src/middleware/validate.ts, src/middleware/errorHandler.ts

validate.ts:

```
import { Request, Response, NextFunction } from 'express';
import { ZodSchema } from 'zod';

export function validate(schema: ZodSchema) {
  return (req: Request, _res: Response, next: NextFunction): void => {
    schema.parse(req.body);
    next();
  };
}
```

errorHandler.ts:

```
import { ZodError } from 'zod';

export function errorHandler(err: Error, _req, res, _next): void {
  if (err instanceof ZodError) {
    res.status(400).json({
      error: 'Помилка валідації',
      details: err.issues.map(i => ({ field: i.path.join("."), message: i.message
})),
    });
    return;
  }
  res.status(500).json({ error: "Внутрішня помилка сервера" });
}
```

routes/recipe.ts — основні маршрути:

```
// GET /recipes/quick — швидкі рецепти
router.get('/quick', (_req, res) => { res.json(getQuickRecipes(30)); });

// GET /recipes — всі з фільтрацією
router.get('/', (req, res) => {
  const filters = {};
  if (req.query.difficulty) filters.difficulty = String(req.query.difficulty);
  if (req.query.cuisine) filters.cuisine = String(req.query.cuisine);
  if (req.query.maxCookingTime) filters.maxCookingTime = Num-
ber(req.query.maxCookingTime);
  if (req.query.search) filters.search = String(req.query.search);
  res.json(getAllRecipes(filters));
});

// GET /recipes/:id
router.get('/:id', (req, res) => {
  const recipe = getRecipeById(req.params.id);
  if (!recipe) { res.status(404).json({ error: 'Рецепт не знайдено' }); return; }
  res.json(recipe);
});

// POST /recipes
router.post('/', validate(createRecipeSchema), (req, res) => {
  res.status(201).json(createRecipe(req.body));
});

// PATCH /recipes/:id
router.patch('/:id', validate(updateRecipeSchema), (req, res) => {
  const recipe = updateRecipe(req.params.id, req.body);
  if (!recipe) { res.status(404).json({ error: 'Рецепт не знайдено' }); return; }
  res.json(recipe);
});

// DELETE /recipes/:id
router.delete('/:id', (req, res) => {
  if (!deleteRecipe(req.params.id)) { res.status(404).json({ error: "Не знайдено" });
return; }
  res.status(204).send();
```

```
});
```

## Завдання 5: Розділення app.ts та server.ts

app.ts — створює і налаштовує Express без listen():

```
import express from 'express';
import cors from 'cors';
import recipeRouter from './routes/recipe';
import { errorHandler } from './middleware/errorHandler';

const app = express();
app.use(cors());
app.use(express.json());
app.use('/api/recipes', recipeRouter);
app.use(errorHandler);
export default app;
```

server.ts — лише запуск сервера:

```
import app from './app';
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => { console.log(`Сервер на http://localhost:${PORT}`); });
```

## Завдання 6: Інтеграційні тести

Файл: tests/recipe.test.ts. Написано 22 інтеграційних тести з використанням Supertest. Тести покривають всі маршрути, включаючи граничні випадки.

```
import request from 'supertest';
import { Server } from 'http';
import app from '../src/app';
import { clearStorage } from '../src/storage/recipe';

let server: Server;
beforeAll(() => { server = app.listen(0); });
afterAll((done) => { server.close(done); });
beforeEach(() => { clearStorage(); });

describe('POST /api/recipes', () => {
  it('створює рецепт і повертає 201', async () => {
    const res = await request(app).post('/api/recipes').send(validRecipe).expect(201);
    expect(res.body).toHaveProperty('id');
  });

  it('повертає 400 якщо title відсутній', async () => {
    await request(app).post('/api/recipes').send({}).expect(400);
  });
  // ... ще 20 тестів
});
```

## Завдання 7: Фільтрація через query-параметри

Реалізовано 4 параметри фільтрації: ?difficulty=easy, ?cuisine=italian, ?maxCooking-Time=30, ?search=mushroom. Параметри можна комбінувати.

Спеціальний маршрут GET /api/recipes/quick повертає рецепти до 30 хвилин.

```
// Фільтрація реалізована на рівні сховища:
export function getAllRecipes(filters?: RecipeFilters): Recipe[] {
  if (filters?.difficulty) recipes = recipes.filter(r => r.difficulty === fil-
ters.difficulty);
  if (filters?.cuisine) recipes = recipes.filter(r => r.cuisine === filters.cuisine);
  if (filters?.maxCookingTime) recipes = recipes.filter(r => r.cookingTimeMinutes <=
filters.maxCookingTime!);
  if (filters?.search) recipes = recipes.filter(r =>
r.title.toLowerCase().includes(term));
}
```

## Результати тестування

Запуск: npm test

```
PS E:\2 курс\Node.Js\lab3> npm test

> lab3-recipes-api@1.0.0 test
> jest

 PASS  tests/recipe.test.ts
  POST /api/recipes
    √ створює рецепт і повертає 201 (91 ms)
    √ повертає 400 якщо title відсутній (12 ms)
    √ повертає 400 якщо title порожній рядок (8 ms)
    √ повертає 400 якщо difficulty невалідний (8 ms)
    √ повертає 400 якщо cookingTimeMinutes менше 1 (8 ms)
    √ створює рецепт без опціональних полів (8 ms)
  GET /api/recipes
    √ повертає порожній масив якщо немає рецептів (7 ms)
    √ повертає всі рецепти (18 ms)
    √ фільтрує за difficulty (18 ms)
    √ фільтрує за cuisine (16 ms)
    √ фільтрує за maxCookingTime (17 ms)
    √ фільтрує за пошуковим запитом search (18 ms)
    √ комбінує кілька фільтрів (21 ms)
  GET /api/recipes/:id
    √ повертає рецепт за id (8 ms)
    √ повертає 404 якщо рецепт не існує (4 ms)
  PATCH /api/recipes/:id
    √ оновлює рецепт і повертає оновлені дані (10 ms)
    √ повертає 404 при оновленні неіснуючого рецепту (5 ms)
    √ повертає 400 якщо дані невалідні (12 ms)
  DELETE /api/recipes/:id
    √ видаляє рецепт і повертає 204 (15 ms)
    √ повертає 404 при видаленні неіснуючого рецепту (5 ms)
  GET /api/recipes/quick
    √ повертає лише рецепти до 30 хвилин (20 ms)
    √ повертає порожній масив якщо немає швидких рецептів (12 ms)

Test Suites: 1 passed, 1 total
Tests:       22 passed, 22 total
Snapshots:   0 total
Time:        4.823 s
ab3 > tests > recipe.test.ts
```

## Висновок

У ході виконання лабораторної роботи було реалізовано RESTful API для управління рецептами на основі Express.js з TypeScript. Використано Zod для валідації вхідних даних, Map для in-memory зберігання, Supertest для інтеграційного тестування. Реалізовано повний набір CRUD-операцій, middleware для валідації та обробки помилок, фільтрацію за кількома параметрами та спеціальний маршрут для швидких рецептів.